3789. Minimum Cost to Acquire Required Items

Daily Question - November 11, 2026

Problem Statement

You are given five integers cost1, cost2, costBoth, need1, and need2.

There are three types of items:

An item of type 1 costs cost1 and contributes 1 unit to the type 1 requirement only. An item of type 2 costs cost2 and contributes 1 unit to the type 2 requirement only. An item of type 3 costs costBoth and contributes 1 unit to both type 1 and type 2 requirements.

You must collect enough items so that the total contribution toward type 1 is at least need1 and the total contribution toward type 2 is at least need2. Return the minimum possible total cost.

Examples

Example 1: cost1 = 3, cost2 = 2, costBoth = 1, need1 = 3, need2 = 2. Output: 3. Buy three type 3 items for a total cost of 3; this satisfies both requirements.

Example 2: cost1 = 5, cost2 = 4, costBoth = 15, need1 = 2, need2 = 3. Output: 22. Buy two type 1 items and three type 2 items: 2 * 5 + 3 * 4 = 22.

Example 3: cost1 = 5, cost2 = 4, costBoth = 15, need1 = 0, need2 = 0. Output: 0. No items are needed.

Constraints

1 <= cost1, cost2, costBoth <= 1000000; 0 <= need1, need2 <= 1000000000.

Solution: Case Analysis

There are only three useful purchasing strategies:

1. Buy type 1 and type 2 items separately: separateOnly = need1 * cost1 + need2 * cost2.

2. Buy only type 3 items: combinedOnly = costBoth * max(need1, need2).

3. Buy min(need1, need2) type 3 items to cover the shared requirement, then buy separate items for the remaining requirement: mixed = common * costBoth + (need1 - common) * cost1 + (need2 - common) * cost2, where common = min(need1, need2).

Return the minimum of these three values. Use 64-bit arithmetic because need1 and need2 can be as large as 10^9 and item costs can be as large as 10^6.

Complete Java Solution

```
class Solution {
    public long minimumCost(
            int cost1,
```

```
            int cost2,
            int costBoth,
            int need1,
            int need2) {

        long separateOnly =
                (long) need1 * cost1
                + (long) need2 * cost2;

        long combinedOnly =
                (long) costBoth * Math.max(need1, need2);

        long common = Math.min(need1, need2);
        long mixed =
                (long) common * costBoth
                + (long) (need1 - common) * cost1
                + (long) (need2 - common) * cost2;

        return Math.min(separateOnly, Math.min(combinedOnly, mixed));
    }
}
```

Correctness

If no type 3 item is used, the cheapest way is to buy exactly need1 type 1 items and need2 type 2 items, giving separateOnly. If every required unit is supplied by type 3 items, the number needed is max(need1, need2), giving combinedOnly.

For any mixed solution, let common be the number of type 3 items used. It cannot usefully exceed min(need1, need2), because an additional type 3 item would provide an unnecessary unit for at least one requirement. For a fixed number of type 3 items, the remaining requirements are best satisfied by exactly the needed number of type 1 and type 2 items. Thus the mixed formula with common = min(need1, need2) represents the best possible mixed plan.

These three candidates cover zero type 3 items, the maximum useful number of type 3 items, and the optimal intermediate number. Therefore, their minimum is the global minimum.

Complexity

Time: O(1). Auxiliary space: O(1).

Implementation Notes

Cast operands to long before multiplication to prevent int overflow. For example, need1 * cost1 can reach 10^15, which exceeds the 32-bit integer range. The return type is long. If need1 and need2 are both zero, all three formulas evaluate to 0, so the method correctly returns 0.